

lock.lock为什么写到try语句外？

面试官：小伙子，JUC 并发包下的可重入锁 ReentrantLock 在代码里实际使用过么。

混子：用过，**ReentrantLock 是 JDK 提供的可重入的锁**。提供对 **共享资源的独占访问**，一次只能有一个线程可以获取该锁。

面试官：你觉得，**ReentrantLock#lock** 方法写到 **try** 语句外面还是里面。

混子：我.....

面试官：我们不合适，你走吧。

先给出结论，lock.lock() 最规范的写法是写到 try 语句的外面。

lock.lock()

默认小伙伴对 ReentrantLock 和 AQS 相关的知识是掌握的。

Oracle 文档中在介绍锁的使用时有一段代码，我们以 **ReentrantLock 举例**，代码如下所示：

```
ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
    // access the resource protected by this lock
} finally {
    lock.unlock();
}
```

Q：为什么要把 **lock.unlock()** 放到 finally 语句块？

A：为了保证当前线程执行过程中出现异常时，锁依然能被释放掉，**避免死锁的产生**。

我们来改动一下上面的代码，看看会产生什么样的影响。

```
ReentrantLock lock = new ReentrantLock();
try {
    lock.lock();
    // access the resource protected by this lock
} finally {
    lock.unlock();
}
```

看着没问题呀，为啥文章开始不建议这么用？先说下可能会存在的问题。

异常堆栈丢失

假设在 `lock.lock` 方法中加锁异常（千万不要杠），那么会进入 `finally` 语句块中进行解锁。

继续跟进，看一下 `lock.unlock()` 源码中是如何处理的。

```
protected final boolean tryRelease(int releases) {
    int c = getState() - releases;
    if (Thread.currentThread() != getExclusiveOwnerThread())
        throw new IllegalMonitorStateException();
    boolean free = false;
    if (c == 0) {
        free = true;
        setExclusiveOwnerThread(null);
    }
    setState(c);
    return free;
}
```

`lock.lock()` 抛出异常有可能还没获取到锁，那么 **解锁源码中将当前线程比较拥有锁线程肯定是不相等的**，所以会抛出 `IMSE`（`IllegalMonitorStateException`）异常。

我重写了 `ReentrantLock` 加锁代码的逻辑，在里面抛出了异常，一起看下会出现什么情况：

```
final void lock() {
    // 模拟加锁未成功就抛出异常
    if (true) {
        throw new RuntimeException("报错啦!!!");
    }
    if (compareAndSetState(0, 1))
```

```

        setExclusiveOwnerThread(Thread.currentThread());
    else
        acquire(1);
}

```

根据下图可以看出 **加锁时异常堆栈被"吞掉了"**，悄无声息的就没了。当然这只是举例，但是谁能保证加锁未成功时不会抛出异常呢。

```

public static void main(String[] args) {
    Lock lock = new SupportReentrantLock();
    try {
        lock.lock();
    } finally {
        lock.unlock();
    }
}

```

ReentrantLockDemo

Exception in thread "main" java.lang.IllegalMonitorStateException
 at cn.hsa.cep.tpc.bdc.SupportReentrantLock\$Sync.tryRelease([ReentrantLockSupport.java:61](#)) <1 internal call>
 at cn.hsa.cep.tpc.bdc.SupportReentrantLock.unlock([ReentrantLockSupport.java:369](#))
 at cn.hsa.cep.tpc.bdc.ReentrantLockDemo.main([ReentrantLockDemo.java:52](#))

Process finished with exit code 1

真实存在的 BUG

上面代码示例中都是在 try 的第一行写 lock，给大家提供一个反面教材，千万千万不要有这种类似行为。

```

public static void main(String[] args) {
    ReentrantLock lock = new ReentrantLock();
    try {
        int i = 1 / 0;
        lock.lock();
    } finally {
        lock.unlock();
    }
}

```

ReentrantLockDemo > main()

Exception in thread "main" java.lang.IllegalMonitorStateException
 at java.util.concurrent.locks.ReentrantLock\$Sync.tryRelease([ReentrantLock.java:151](#))
 at java.util.concurrent.locks.AbstractQueuedSynchronizer.release([AbstractQueuedSynchronizer.java:1261](#))
 at java.util.concurrent.locks.ReentrantLock.unlock([ReentrantLock.java:457](#))
 at cn.hsa.cep.tpc.bdc.ReentrantLockDemo.main([ReentrantLockDemo.java:52](#))

Process finished with exit code 1

示例代码中把 lock 放到了 try 语句块里，然后 **lock 加锁前面还有可能会产生异常的代码**，这种就凉了，谁用谁凉的那种。

文末总结

所以关于要不要把 **lock.lock()** 写到 **try 语句块里**，文章的结论是：

1. 最好是把 lock.lock() 加锁方法写到 try 外面，**这是一种规范，而不是强制**。
2. 如果你非要写到 try 里面，那么 **请写到 try 语句块的第一行**，或者 lock 加锁方法前面不会存在可能出现异常的代码。
3. 最后，**如果你代码中加锁放到了 try 语句里，麻烦参考第 1 点**。

更新: 2023-07-22 14:47:19

原文: <<https://www.yuque.com/magestack/12306/pu52u29i6eb1c5wh>>